RedFlag is a monitor utility for the people in the sales department. It should send the sales folks a message when an order is late, when total sales are too low…basically, whenever one of many different things happens. Sales wants to monitor dozens of different events, and the list is bound to change every week or so.

Luckily for you and Bill, sales has full-time programmers, so you don't have to write the events yourselves. You can just write a simple Domain-Specific Language. (You can read about DSLs in Appendix 2, *Domain-Specific Languages*, on page 227.) The sales guys can then use this DSL to define events, like this:

```
event "we're earning wads of money" do
  recent_orders = ...   # (read from database)
  recent_orders > 1000
end
```

To define an event, you give it a description and a block of code. If the block returns true, then you get an alert via mail. If it returns false, then nothing happens. The system should check all the events every few minutes.

It's time to write RedFlag 0.1.

Your First DSL

You and Bill put together a working RedFlag DSL in no time:

blocks/redflag_1/redflag.rb

```
def event(description)
  puts "ALERT: #{description}" if yield
end
load 'events.rb'
```

The entire DSL is just one method and a line that executes a file named events.rb. The code in events.rb is supposed to call back into RedFlag's event method. To test the DSL, you create a quick events file:

blocks/redflag_1/events.rb

```
event "an event that always happens" do
  true
end
event "an event that never happens" do
  false
end
```

You save both redflag.rb and events.rb in the same folder and run redflag.rb:

```
❮ ALERT: an event that always happens
```